

Final Progress Report — AI RAG Tutor

Sachet Kanchugar | Akash Deep Singh | Sanjeet Srinivasa
Foundations of AI | March 30, 2026

What Has Already Been Completed

The project implements an **AI RAG tutor** that allows teachers to upload course documents and students to interact with a chatbot. The AI chatbot answers student questions based solely on the uploaded course materials, providing citations when relevant.

Frontend:

- Built using **React 18**, with **React Router DOM** for navigation and **Axios** for API communication.
- Styled with **TailwindCSS** for a responsive, user-friendly interface.
- Complete authentication flow implemented:
 - Sign up, sign in, role selection, and protected routing.
 - Students can only access the chatbot interface.
 - Teachers have access to both chatbot and document upload/management interfaces.
- Modular component structure includes `App.js`, `ChatPage.js`, `UploadPage.js`, authentication pages, and role-selection pages.
- All API calls are integrated with error handling and state management.
- Modern, responsive UI with loading states and error messages for smoother interaction.
- Fully integrated with backend services for seamless chat, document upload, and document retrieval.

Backend:

- Built with **FastAPI**, fully modularized with separate modules for authentication, database, RAG pipeline, vector store, and password utilities.
- User authentication and role management implemented using JWT and secure password hashing (`bcrypt`).
- Endpoints for document upload, listing, and chat Q&A are implemented and protected.
- Authentication system enforces secure, role-based access:
 - Both students and teachers must authenticate to access the system.
 - Teachers automatically have upload permissions; role-based access is enforced based on the user role.
- Core workflows—document upload, retrieval, and chat—are operational and integrated through the React frontend.
- Error handling and state management ensure a reliable user experience.

RAG Pipeline:

- Uploaded documents (PDF, TXT, MD) are parsed, chunked, embedded, and indexed in a local **FAISS** vector database.
- Document metadata such as uploader, timestamp, and type is tracked for each chunk.
- Chunking uses `RecursiveCharacterTextSplitter` (chunk size = 500, overlap = 100).
- Embeddings are generated using `text-embedding-3-small`.
- RAG pipeline retrieves top-3 relevant chunks and generates answers with citations (document name, page, snippet) using `gpt-4o-mini`.

Overall, the **AI RAG tutor** is **fully functional**: students can chat with the AI, teachers can upload documents, and responses are grounded in uploaded materials.

What Remains Before Final Submission

The remaining tasks are primarily **quality-of-life improvements and minor polishing**:

- **Frontend enhancements**: minor UI polish, chat auto-scroll, improved loading indicators, and smoother error messaging.
- **Backend improvements**: logging for uploads and chat queries, additional error handling for edge cases, optional FAISS index persistence for faster restarts.
- **Testing**: end-to-end testing with multiple documents and user roles to ensure stability and correct retrieval/citation behavior.

No major new features are required; all core functionality of the **AI RAG tutor** is complete. The next steps focus on improving **usability, robustness, and final testing**.